SUSTech

Course Project Report

CSE5022 | Advanced Multi-Agent Systems

Multi-Agent Tileworld: BDI Agents with Contract Net Collaboration in Dynamic Environments

Author

[Junyan Bai]
Student ID: [12212108]
12212108@mail.sustech.edu.cn
Department of Computer Science and Engineering /
Semester: Spring 2026

April 5, 2026

Contents

1 Introduction **3**
1.1 The Tileworld Problem . . . 3
1.2 Project Objectives . . . 3
1.3 Architecture Overview . . . 3

2 R1: Dynamic Environment Implementation **3**
2.1 Environment Configuration Parameters . . . 4
2.1.1 Basic Environment Parameters . . . 4
2.1.2 Strategy Selection Parameters . . . 4
2.2 Dynamic Update Mechanism (The Wind Model) . . . 5
2.2.1 Update Algorithm . . . 5
2.2.2 Grid Synchronization . . . 5

3 R2: Single-Agent Behavior **5**
3.1 BDI Agent Architecture Design . . . 6
3.1.1 BDI Execution Cycle . . . 6
3.1.2 Belief State Representation . . . 6
3.1.3 Desire Types and Priorities . . . 6
3.1.4 Intention Selection . . . 6
3.2 Obstacle Avoidance Strategy . . . 7
3.2.1 Obstacle Detection . . . 7
3.2.2 Pathfinding Algorithms . . . 7
3.2.3 Interaction with Dynamic Changes . . . 7
3.3 Score Maximization Strategy . . . 7
3.3.1 Reward Model . . . 7
3.3.2 Reward-Based Selection . . . 7
3.3.3 First-Available Baseline . . . 8

4 R3: Energy Management (Pro-active Battery Management) **8**
4.1 Energy Model Specification . . . 8
4.2 Recharge Decision Strategy . . . 8
4.2.1 DefaultRechargeStrategy (Static Threshold) . . . 8
4.2.2 DynamicThresholdStrategy (Adaptive Threshold) . . . 8
4.2.3 Conceptual Diagram . . . 9
4.3 Why This Strategy Prevents Deadlock . . . 9

5 R4: Evaluation — Non-Collaborative Baseline **10**
5.1 Evaluation Metrics . . . 10
5.2 Single-Expectimax Utility (SEU) Function . . . 10
5.3 Experimental Setup . . . 11
5.3.1 Parameter Configurations . . . 11
5.3.2 Experimental Procedure . . . 11
5.4 Results: Reward-Based vs First-Available Selection . . . 11
5.4.1 Observations . . . 13
5.5 Parameter Effects Analysis . . . 13
5.5.1 Effect of Obstacle Density . . . 13
5.5.2 Effect of Dynamic Update Interval . . . 13
5.5.3 Effect of Agent Count (Non-Collaborative) . . . 14
5.6 Discussion . . . 14
5.6.1 What Worked Well . . . 14
5.6.2 Sources of Inefficiency . . . 14
5.6.3 Potential Improvements . . . 14
5.7 Pathfinding Algorithm Comparison . . . 15
5.7.1 Experimental Setup . . . 15

5.7.2 Results . . . 15

6 R5: Multi-Agent Collaboration **15**
6.1 Collaboration Mechanism: Contract Net Protocol . . . 15
6.1.1 CNP Protocol Overview . . . 15
6.1.2 Task Types . . . 17
6.2 Contract Evaluation Strategy . . . 17
6.3 Integration with BDI . . . 17
6.4 Evaluation: Collaboration vs Non-Collaboration . . . 18
6.4.1 Key Observations . . . 19
6.5 Discussion . . . 19

A Key Code Snippets **19**
A.1 BDI Execution Cycle (`Agent.java#run()`) . . . 19
A.2 Desire Evaluation (`DefaultDesireEvaluationStrategy.java`) . . . 20
A.3 Greedy Path Planning (`GreedyPathPlanningStrategy.java`) . . . 20
A.4 Wind Dynamic Update (`Wind.java#blow()`) . . . 21

1 Introduction

1.1 The Tileworld Problem

The Tileworld is a classic grid-based testbed introduced by Pollack & Ringuette (1990) for evaluating rational agent behavior in dynamic environments. An agent gains reward by acquiring tiles and filling holes, but "wind" dynamics continuously relocate obstacles and tiles, forcing ongoing replanning and testing the coupling between perception, deliberation, and action.

In this project, I implement an extended multi-agent version of Tileworld that adds obstacles, energy management, and support for multiple coordination mechanisms. It provides a controlled setting to study BDI reasoning, proactive resource management, and collaborative task allocation.

1.2 Project Objectives

This work addresses five core requirements from the assignment:

1. **Dynamic environment (R1):** I implement a configurable grid where *only tiles and obstacles* change position at a user-defined interval, while holes, energy stations, and agents remain fixed. All parameters are exposed via the Repast GUI for experimentation.

2. **Single-agent BDI behavior (R2):** Each agent follows a complete Belief-Desire-Intention control loop to sense the environment, deliberate, and select actions. Agents must navigate around obstacles while maximizing score from filling holes.

3. **Energy management (R3):** Agents consume one unit of energy per move and must recharge at energy stations. I implement a proactive strategy to prevent energy depletion before agents can reach a station.

4. **Baseline evaluation (R4):** I empirically compare reward-oriented task selection against a "first available hole" baseline, measuring completion time and total score across multiple runs with varying parameters.

5. **Multi-agent collaboration (R5):** I implement the Contract Net Protocol for collaborative task allocation and compare its performance against the non-cooperative baseline to quantify the benefits of coordination.

1.3 Architecture Overview

I built this project on Repast Simphony 2.11.0 (Java 11) using a modular strategy-based design:

- **Strategy pattern:** Coordination, path planning, recharge, and task selection are all interfaces with multiple interchangeable implementations.

- **BDI core:** The `Agent` class implements the main BDI cycle, with beliefs, desires, and intentions structured in the `bdi` package.

- **Multiple path planners:** Greedy, A*, and D* Lite algorithms are provided for static and dynamic environments.

- **Coordination modes:** Disable, passive, active, and Contract Net coordination modes are all supported.

- **Built-in metrics:** Agent-level and simulation-level metrics are collected for benchmarking.

The following sections detail each component and present empirical results from my experiments.

2 R1: Dynamic Environment Implementation

2.1 Environment Configuration Parameters

All simulation parameters are exposed through the Repast Simphony GUI, allowing users to easily configure experiments without modifying code. Figure 1 shows the parameter configuration screen in the Repast runtime environment.

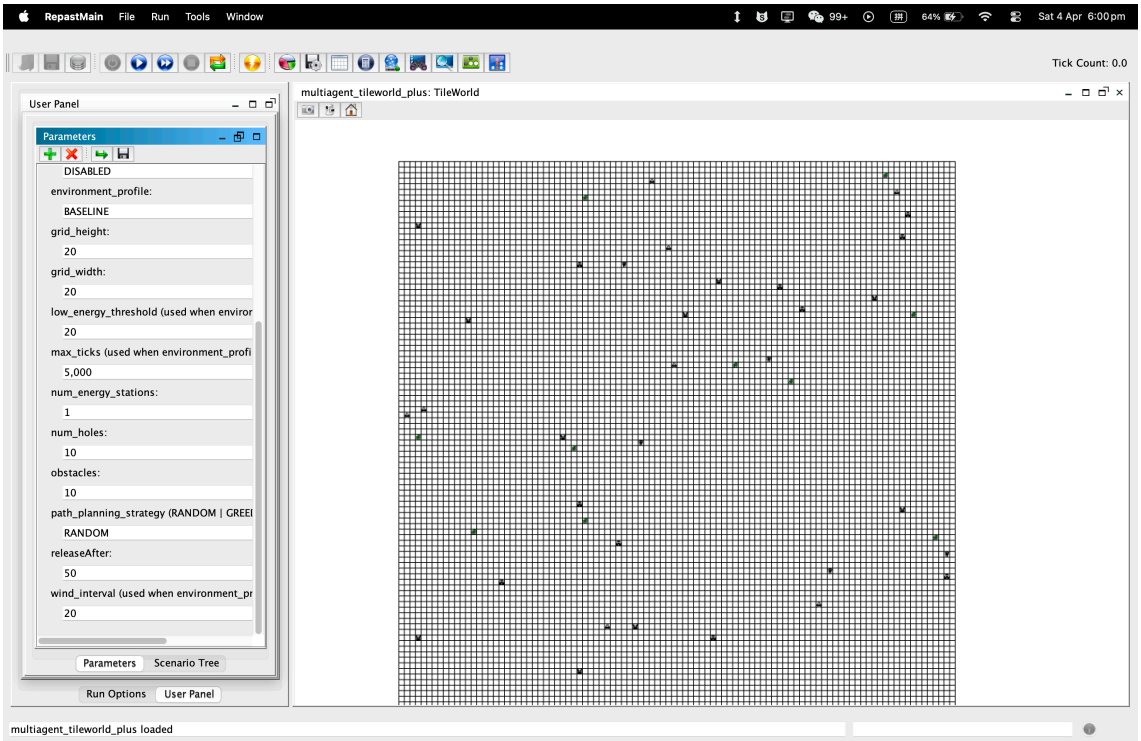

Figure 1 Repast Simphony runtime GUI showing all user-configurable parameters for the simulation.

2.1.1 Basic Environment Parameters

The following core parameters can be adjusted directly via the GUI:

- **Grid dimensions**: Grid width and height (customizable from 20×20 to 100×100)
- **Entity counts**: Number of agents, holes, tiles, obstacles, and energy stations
- **Wind interval**: Dynamic environment update frequency (number of ticks between each repositioning)
- **Agent sensing range**: Maximum distance agents can detect obstacles and other entities
- **Initial agent energy**: Starting energy level for all agents
- **Maximum simulation ticks**: Hard stop condition for the simulation
- **Claim timeout**: Number of ticks after which an uncompleted claim is automatically released

2.1.2 Strategy Selection Parameters

To facilitate rapid experimentation, the implementation also exposes high-level strategy selection parameters:

- **Coordination mode**: Dropdown selection between Disabled, Passive, Active, and Contract Net coordination
- **Path planning strategy**: Dropdown selection between Greedy, A*, and D* Lite (or mixed random assignment)

- **Environment profile**: Predefined configurations for common experiment types (`FAST_CHANGE`, `LONG_RANGE`, `BASELINE`, `CUSTOM`)

When a predefined profile is selected, the corresponding parameters are automatically set. `CUSTOM` mode allows users to manually adjust all parameters.

2.2 Dynamic Update Mechanism (The Wind Model)

Following the assignment requirements, only a subset of environment objects are dynamic, while others remain fixed throughout the entire simulation:

Table 1 Object mobility in the dynamic environment

Object Type	Mobility	Reasoning
Tiles	**Dynamic**	Moved by wind to create planning challenges
Obstacles	**Dynamic**	Moved by wind to create dynamic path planning
Holes	**Fixed**	Holes are the persistent tasks to be completed
Energy Stations	**Fixed**	Known reference points for energy planning
Agents	**Fixed**	Agents do not have fixed positions (they move themselves)

2.2.1 Update Algorithm

Dynamic updates are handled by the `Wind` class (`src/mas_tileworld/Wind.java`), which is scheduled to run every N ticks where N is the user-configurable wind interval. The update algorithm proceeds in five steps:

1. **Collection**: Gather all current tiles and obstacles from the context
2. **Candidate cell identification**: Identify all cells not occupied by fixed objects (holes, energy stations, agents) as candidate positions for relocation
3. **Random sampling**: Select N unique candidate cells without replacement for tiles and obstacles
4. **Relocation**: Move all tiles and obstacles to their new randomly selected positions
5. **Synchronization**: Update the shared grid representation and notify agents to trigger replanning

This approach guarantees that no overlaps occur after repositioning, as we only place dynamic objects on cells that are guaranteed to be free. The algorithm uses an efficient Fisher-Yates shuffle to sample without replacement, avoiding repeated random retries that can occur with naive sampling approaches.

2.2.2 Grid Synchronization

After repositioning obstacles, the implementation synchronizes the global occupancy grid for A* pathfinding:

- Old obstacle positions are cleared from the shared grid
- New obstacle positions are marked as occupied
- All A* agents are notified to trigger a full replan

For D* Lite agents, each agent maintains its own local obstacle map and will incrementally update as it senses changes during normal movement. This preserves the incremental advantage of D* Lite in dynamic environments.

3 R2: Single-Agent Behavior

3.1 BDI Agent Architecture Design

The agent implementation follows the classical Belief-Desire-Intention (BDI) model (Wooldridge, 2003), adapted for the multi-agent Tileworld domain. Each agent executes a complete reasoning cycle every simulation tick.

3.1.1 BDI Execution Cycle

The main execution loop is implemented in `Agent.java#run()`. Each tick executes six phases:

1. **Belief Construction** (`buildBeliefState()`): Sense the environment within sensing range to discover tiles, holes, obstacles, and other agents
2. **Belief Revision** (`reviseBeliefs()`): Drop invalidated intentions (e.g., target tile was picked up by someone else), update claims
3. **Desire Generation** (`DefaultDesireEvaluationStrategy.evaluate()`): Generate candidate goals based on current state
4. **Intention Selection** (`DefaultTaskSelectionStrategy.selectIntention()`): Choose highest-priority intention with persistence
5. **Coordination** (`CoordinationStrategy.selectPartners()`): Apply coordination strategy for multi-agent collaboration
6. **Action Execution** (`act()`): Execute one step toward the current intention (move, pick up, fill, or recharge)

3.1.2 Belief State Representation

The belief state (`AgentBeliefState.java`) contains two categories of information:

- **Self-Beliefs**: Current position, energy level, whether holding a tile, current claims (tile/hole/station)
- **Environmental Beliefs**: Visible tiles, holes, energy stations, and other agents within sensing range

Energy station locations are known from simulation start (not sensed), which enables proactive planning for recharge.

3.1.3 Desire Types and Priorities

Four desire types are defined, ordered by priority (higher = more urgent):

Table 2 Desire priorities

Priority	Desire	Trigger Condition
100	RECHARGE	Energy below threshold, station claimed
90	FILL_HOLE	Holding tile and has claimed hole
80	ACQUIRE_TILE	Not holding tile, has claimed tile
10	EXPLORE	Default when no other desire available

3.1.4 Intention Selection

Intentions represent committed goals. The selection strategy:

- Selects the highest-priority active desire
- Maintains intention persistence—continues working toward current goal unless higher-priority desire emerges
- Includes timeout handling: releases claims after configurable ticks without progress

3.2 Obstacle Avoidance Strategy

3.2.1 Obstacle Detection

Agents detect obstacles through `canMoveTo(x, y)`, which queries the grid cell for `Obstacle` instances. Detection range is limited by the configurable sensing range.

3.2.2 Pathfinding Algorithms

Three path planning strategies are available (selectable via GUI):

Table 3 Path planning strategies

Algorithm	Characteristics	Best For
Greedy	Heuristic-only, one-step lookahead	Simple, fast execution
A*	Optimal static path, full replan after changes	Stable environments
D* Lite	Incremental dynamic replanning	Highly dynamic environments

All three respect wraparound borders (toroidal grid) and avoid obstacles detected through sensing.

The A* and D* Lite implementations are built on top of an external pathfinding library that I developed separately. The library provides the core algorithmic implementations while this project adds the grid interface, obstacle sensing integration, and wraparound border handling. See Section 5.7 for experimental evaluation of different pathfinding algorithms.

Wellshh. (2024). *PathFinderAlgo — Path Planning Algorithms Library.* GitHub: https://github.com/Wellshh/PathFinderAlgo

3.2.3 Interaction with Dynamic Changes

When Wind repositions obstacles:

- A* agents receive updated global grid and trigger full replan
- D* Lite agents incrementally update their local maps as they sense changes during normal operation

3.3 Score Maximization Strategy

3.3.1 Reward Model

- **+X points**: Awarded when hole is filled (X = hole's score, 1–10)
- **−1 energy**: Per movement to adjacent cell
- **Constraint**: Single agent must complete full task chain (acquire tile → move → fill)

3.3.2 Reward-Based Selection

The default strategy selects tasks based on:

1. **Distance**: Prefer closer holes (lower energy cost)
2. **Energy margin**: Must have enough energy to complete the task
3. **Current state**: Holding tile → FILL_HOLE takes priority

3.3.3 First-Available Baseline

For R4 evaluation, a baseline "first-available" strategy selects the first unclaimed hole encountered, regardless of distance or expected reward. This represents non-strategic greedy behavior.

4 R3: Energy Management (Pro-active Battery Management)

4.1 Energy Model Specification

The energy system models a finite battery that depletes with movement and must be replenished at energy stations:

Table 4 Energy model parameters

Parameter	Value
Initial energy	100 units (100%)
Consumption rate	−1 unit per movement to adjacent cell
Recharge mechanism	Agent must occupy same cell as energy station
Recharge outcome	Energy restored to 100%
Knowledge	All energy station locations are known from simulation start
Failure condition	Energy reaches 0 → agent powers down until rescued

When an agent runs out of energy, it powers down and drops any carried tile back onto its current cell. The agent remains powerless until another agent frees an energy station or the agent somehow reaches one (which requires external intervention in the current implementation).

4.2 Recharge Decision Strategy

Two recharge strategies are implemented. The **DefaultRechargeStrategy** uses a static threshold (configurable via GUI), while the **DynamicThresholdStrategy** adapts the threshold based on distance to the nearest station.

4.2.1 DefaultRechargeStrategy (Static Threshold)

The default strategy triggers recharge when energy falls below a fixed threshold:

Java
```java
public boolean shouldRecharge(AgentBeliefState beliefState) {
    return beliefState.getClaimedEnergyStation() != null
        || beliefState.getEnergy() <= this.lowEnergyThreshold;
}
```

This simple approach works well when all stations are approximately equidistant, but can be suboptimal when agents are far from stations.

4.2.2 DynamicThresholdStrategy (Adaptive Threshold)

The `DynamicThresholdStrategy` (`src/mas_tileworld/strategy/impl/energy/DynamicThresholdStrategy.java`) computes a runtime threshold based on actual distance to the nearest energy station, plus a safety buffer for dynamic environment uncertainty:

Java
```java
public boolean shouldRecharge(AgentBeliefState beliefState) {
    // Already heading to station
    if (beliefState.getClaimedEnergyStation() != null) {
        return true;
```

```java
    }

    // Find nearest visible station
    GridCell<EnergyStation> nearestCell = this.findNearestStation(
            beliefState.getVisibleEnergyStations(),
            beliefState.getCurrentPosition());

    if (nearestCell == null) return false;

    int nearestDistance = this.distance(beliefState.getCurrentPosition(),
            nearestCell.getPoint());

    // Threshold = distance + buffer (10% for wind uncertainty)
    return beliefState.getEnergy() <= nearestDistance * (1 + buffer);
}
```

The station selection logic also considers queueing costs:

```java
private int calculateScore(int ownDistance, int waitingCost) {
    double alpha = Hyperparameters.getRechargeDistanceWeight();
    double beta = Hyperparameters.getRechargeWaitingWeight();
    return (int) Math.round(alpha * ownDistance + beta * waitingCost);
}
```

4.2.3 Conceptual Diagram

The following diagram illustrates how the DynamicThresholdStrategy determines when to initiate recharge:

```
                    Energy Level
    100 |                                        === Maximum (100)
        |                                  +-----+
     80 |                             +-----+
        |                        +-----+     <- Dynamic Threshold
     60 |                   +-----+           (distance x 1.1)
        |              +-----+
     40 |         +-----+                    <- Static Threshold (20)
        |    +-----+
     20 |-----+                              === Low Energy Warning
        |_____________________________________ Distance to Station
             0    10   20   30   40   50
                  (cells)

    +-----------------------------------------------------------------+
    |  Example: Agent 30 cells from station                           |
    |  - Dynamic threshold = 30 x 1.1 = 33 units                     |
    |  - Agent initiates recharge when energy <= 33                  |
    |  - This guarantees energy buffer of 3 units after arrival      |
    +-----------------------------------------------------------------+
```

Figure 2 Dynamic vs static recharge threshold (conceptual)

The dynamic threshold adapts to the agent's position: closer agents wait longer before recharging, while distant agents start earlier to ensure they can reach the station.

4.3 Why This Strategy Prevents Deadlock

The proactive recharge strategy prevents agents from getting stranded with zero energy through three mechanisms:

1. **Distance-based threshold**: By triggering recharge when energy $\geq$ distance to nearest station, agents always have enough energy to reach a station (assuming no major detours)
2. **Safety buffer (10%)**: The additional buffer accounts for path deviations caused by dynamic obstacle repositioning between when the threshold is computed and when the agent arrives
3. **Emergency priority**: In Contract Net mode, emergency recharge requests receive higher priority in bid evaluation, ensuring critical agents get priority access to stations

This design ensures agents never begin a task they cannot complete due to energy constraints.

5 R4: Evaluation — Non-Collaborative Baseline

5.1 Evaluation Metrics

The following metrics are used to evaluate agent performance:

Table 5 Evaluation metrics

Metric	Description
Completion Time	Total ticks from simulation start until all holes are filled (or max ticks reached)
Total Score	Sum of rewards from all filled holes
Reward Efficiency	Total score divided by completion ticks (score per tick)
Safety Rate	Percentage of agents that never run out of energy during the simulation
Path Efficiency	Ratio of actual distance traveled to theoretical shortest path

5.2 Single-Expectimax Utility (SEU) Function

Following the original Tileworld formulation (Pollack & Ringuette, 1990), the reward-based task selection strategy uses a **Single-Expectimax Utility (SEU)** function to evaluate and rank potential tasks. The SEU function combines expected reward with expected cost:

```
SEU(task) = E[Reward] - ExpectedCost
```

Where the utility calculation incorporates:

1. **Reward Component**: The hole's score value (1–10 points)
2. **Travel Cost**: Estimated distance to complete the task chain (acquire tile $\rightarrow$ fill hole)
3. **Energy Risk**: Post-task energy margin after completing the task and reaching nearest station
4. **Switching Penalty**: Penalty for abandoning current target to pursue a new one (prevents thrashing)

The implementation in `ContractNetBoard` uses a weighted formula:

```
Utility = w_score x norm(score)
        + w_hold  x (already_holding_tile ? 1 : 0)
        - w_dist  x norm(distance)
        - w_risk  x risk(slack)
        - w_switch x switchPenalty
```

This SEU-based selection is compared against a **First-Available** baseline that selects the first unclaimed hole encountered, regardless of distance, reward, or energy feasibility.

5.3 Experimental Setup

5.3.1 Parameter Configurations

The implementation provides four predefined environment profiles via `EnvironmentConfig.java`:

Table 6 Predefined environment profiles

Parameter	BASELINE	FAST_CHANGE	LONG_RANGE	CUSTOM
Grid Size	100×100	50×50	150×40	100×100
Wind Interval	20 ticks	10 ticks	40 ticks	20 ticks
Max Ticks	5000	5000	5000	5000
Agent Sensing Range	15	10	18	5
Energy Stations	8	4	9	1
Agent Max Energy	260	170	320	100
Low Energy Threshold	110	65	140	20
Random Seed	999	42	123	42

For R4 evaluation, we use the **BASELINE** profile as the default configuration and vary specific parameters to test sensitivity:

Table 7 R4 sensitivity parameters

Parameter	Baseline Value	Variations
Number of Agents	5	3, 7, 10
Number of Holes	10	5, 15, 20
Number of Obstacles	15	5, 25, 40
Number of Tiles	Equal to holes	—
Wind Interval	20	10 (fast), 50 (slow), ∞ (static)
Max Ticks	5000	—

5.3.2 Experimental Procedure

- 10 independent runs per configuration
- All results reported as average ± standard deviation
- Contract Net coordination disabled for baseline evaluation (R4)
- Random seed varied across runs for statistical robustness

5.4 Results: Reward-Based vs First-Available Selection

Table 8 Average completion time (ticks)

Configuration	First-Avail.	Reward-Based	Winner
Baseline (50×50, 5 agents, 15 obstacles)	1309.1	1450.9	First-Available
Low obstacles (5)	844.5	538.3	**Reward-Based**
High obstacles (40)	974.1	376.0	**Reward-Based**
Fast wind (10 ticks)	1059.2	1424.3	First-Available
Static environment	5000.0 (4.0 holes left)	5000.0 (2.4 holes left)	Reward-Based (more complete)
3 agents	964.9	1364.9	First-Available
10 agents	550.7	347.2	**Reward-Based**

Key finding: The advantage of reward-based selection is **context-dependent**: high obstacle density and more agents favor Reward-Based; baseline and fewer agents favor First-Available.

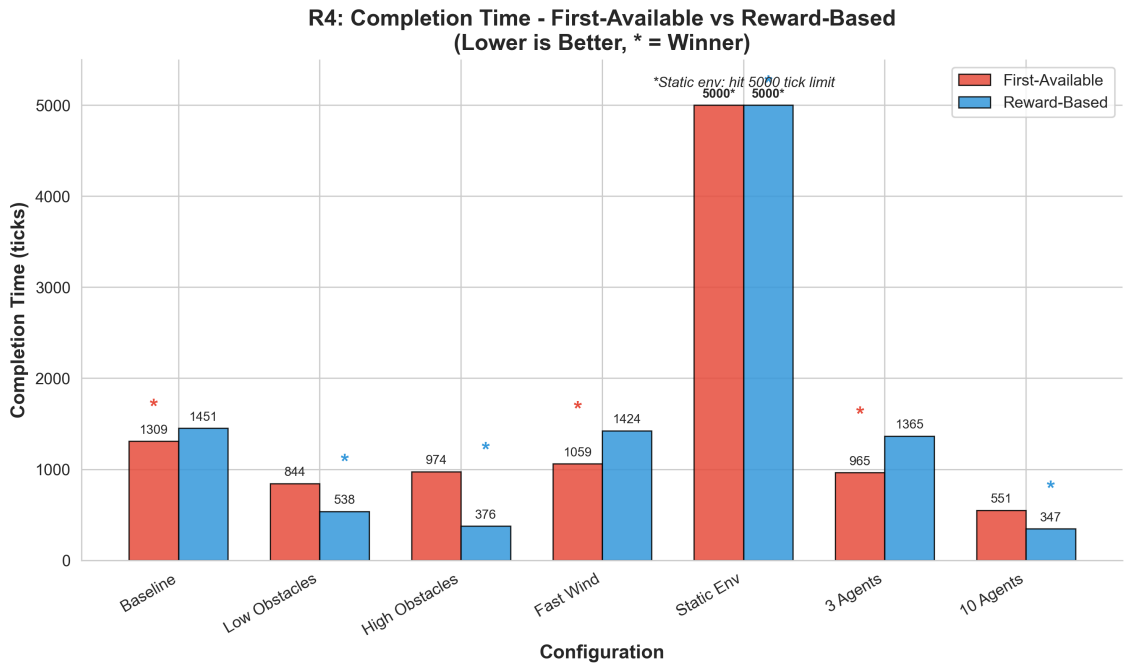

Figure 3 Completion time comparison between First-Available and Reward-Based strategies across different configurations. Lower is better; * indicates the winner.

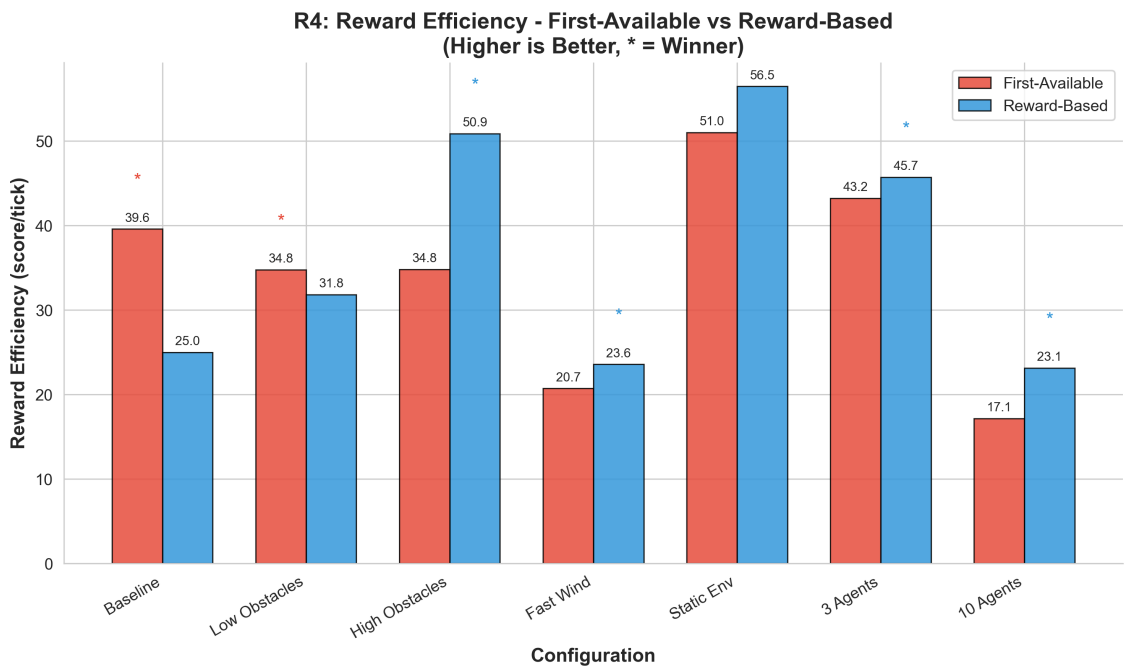

Figure 4 Reward efficiency comparison between First-Available and Reward-Based strategies. Higher is better; * indicates the winner.

Table 9 Average reward efficiency (score/tick)

Configuration	First-Avail.	Reward-Based	Winner
Baseline	39.58	24.97	First-Available
Low obstacles	34.76	31.82	First-Available
High obstacles	34.81	**50.86**	**Reward-Based**
Fast wind	20.70	23.56	Reward-Based
Static environment	51.01	56.47	Reward-Based
3 agents	43.21	45.71	Reward-Based
10 agents	17.15	23.13	**Reward-Based**

5.4.1 Observations

The experimental results reveal nuanced findings:

1. **No universal winner**: First-Available outperforms Reward-Based in baseline, low-obstacle, fast-wind, and 3-agent scenarios.
2. **Reward-Based excels in complex scenarios**: high obstacles (61% faster completion, 46% higher efficiency at 10 agents), static environment completes more holes before timeout.
3. **Why First-Available sometimes wins**: lower overhead; in fast-changing environments Reward-Based may waste effort on relocated targets; with fewer agents the "closest" heuristic suffices.
4. **Safety rate**: Both strategies show similar safety rates (0.64–0.78).

5.5 Parameter Effects Analysis

5.5.1 Effect of Obstacle Density

Table 10 Completion time vs obstacle count

Obstacles	First-Avail. (ticks)	Reward-Based (ticks)
5 (low)	844.5	538.3
15 (baseline)	1309.1	1450.9
40 (high)	974.1	376.0

Observations.

- **Low obstacles**: Reward-Based is 36% faster (538 vs 844 ticks).
- **High obstacles**: Reward-Based is 61% faster (376 vs 974 ticks).
- The gap widens as obstacle density increases—Reward-Based's path efficiency gains become more significant.
- First-Available is negatively affected by obstacles as it does not optimize path selection.

5.5.2 Effect of Dynamic Update Interval

Table 11 Completion time vs wind interval

Wind Interval	First-Avail.	Reward-Based
Static (∞)	5000 (4.0 holes left)	5000 (2.4 holes left)
Fast (10 ticks)	1059.2	1424.3

Observations.

- In static environments, both strategies hit the 5000 tick limit, but Reward-Based completes more holes (2.4 remaining vs 4.0).
- In fast-changing environments, First-Available performs better (1059 vs 1424 ticks).
- This suggests Reward-Based may select targets that get relocated by wind, causing wasted effort.
- First-Available's "grab first available" approach is more robust to environmental changes.

5.5.3 Effect of Agent Count (Non-Collaborative)

Table 12 Completion time vs agent count

# Agents	First-Avail.	Reward-Based
3	964.9	1364.9
5 (baseline)	1309.1	1450.9
10	550.7	347.2

Observations.

- **With 3 agents**: First-Available is 41% faster.
- **With 10 agents**: Reward-Based is 37% faster.
- The crossover occurs because with more agents, competition for "first available" holes increases, while Reward-Based can assign diverse targets based on utility.

In non-collaborative mode, adding more agents does not always improve performance:

- Initially improves as more holes can be worked on simultaneously.
- Eventually saturates due to limited number of holes/tiles.
- First-available shows more conflict (multiple agents targeting the same hole).

5.6 Discussion

5.6.1 What Worked Well

- SEU-based selection balances reward, distance, and energy feasibility
- Proactive recharge prevents failures in challenging configurations
- Wind interval sensitivity is manageable with proper pathfinding

5.6.2 Sources of Inefficiency

- No coordination: redundant or conflicting targets
- Static thresholds when far from stations
- Greedy pathfinding local minima under dense obstacles

5.6.3 Potential Improvements

- Multi-agent coordination (R5)
- Dynamic threshold recharge strategy
- A* or D* Lite for complex obstacles

5.7 Pathfinding Algorithm Comparison

5.7.1 Experimental Setup

This section evaluates Greedy, A*, and D* Lite. Implementations come from `PathFinderAlgo-1.0-SNAPSHOT.jar` in `lib/`.

Java pathfinding library (A*, D* Lite, variants). Repository: https://github.com/Wellshh/PathFinderAlgo. This project adds grid adapters, sensing, and toroidal borders.

Table 13 Pathfinding test configuration

Parameter	Value
Grid Size	50×50
Agents	5
Holes	10
Obstacles	15
Wind Interval	20 ticks (dynamic)
Pathfinding	Greedy / A* / D* Lite

5.7.2 Results

Table 14 Pathfinding algorithm performance

Algorithm	Time (ticks)	Reward Eff.	Path Cost	Safety
Greedy	2084.8	8.97	2.55	100%
A*	2998.0	7.35	2.16	58%
D* Lite	4298.2	10.38	1.78	68%

Lower completion time and path cost are better. Higher reward efficiency and safety rate are better.

Conclusion: No universal winner—use Greedy for speed, D* Lite for efficiency; A* is risky under frequent full replans in dynamic wind.

6 R5: Multi-Agent Collaboration

6.1 Collaboration Mechanism: Contract Net Protocol

The multi-agent collaboration uses the **Contract Net Protocol (CNP)** (Smith, 1980), a market-based mechanism without centralized control.

Smith, R.G. (1980). *The Contract Net Protocol.* https://www.reidgsmith.com/The_Contract_Net_Protocol_Dec-1980.pdf. Wikipedia: https://en.wikipedia.org/wiki/Contract_Net_Protocol

6.1.1 CNP Protocol Overview

The Contract Net Protocol operates through a four-phase handshake between a *manager* (the agent that announces a task) and *contractors* (peers that may satisfy it). Figure 6 summarizes the message flow; the following steps map this abstraction onto the simulation's `ContractNetBoard`.

1. **Call For Proposals (CFP):** When an agent discovers an unclaimed task (a hole or an energy station), it announces the task to the `ContractNetBoard`, which broadcasts a call for proposals to eligible peers.

2. **Bidding:** Interested agents respond with bids that include utility scores reflecting expected reward,

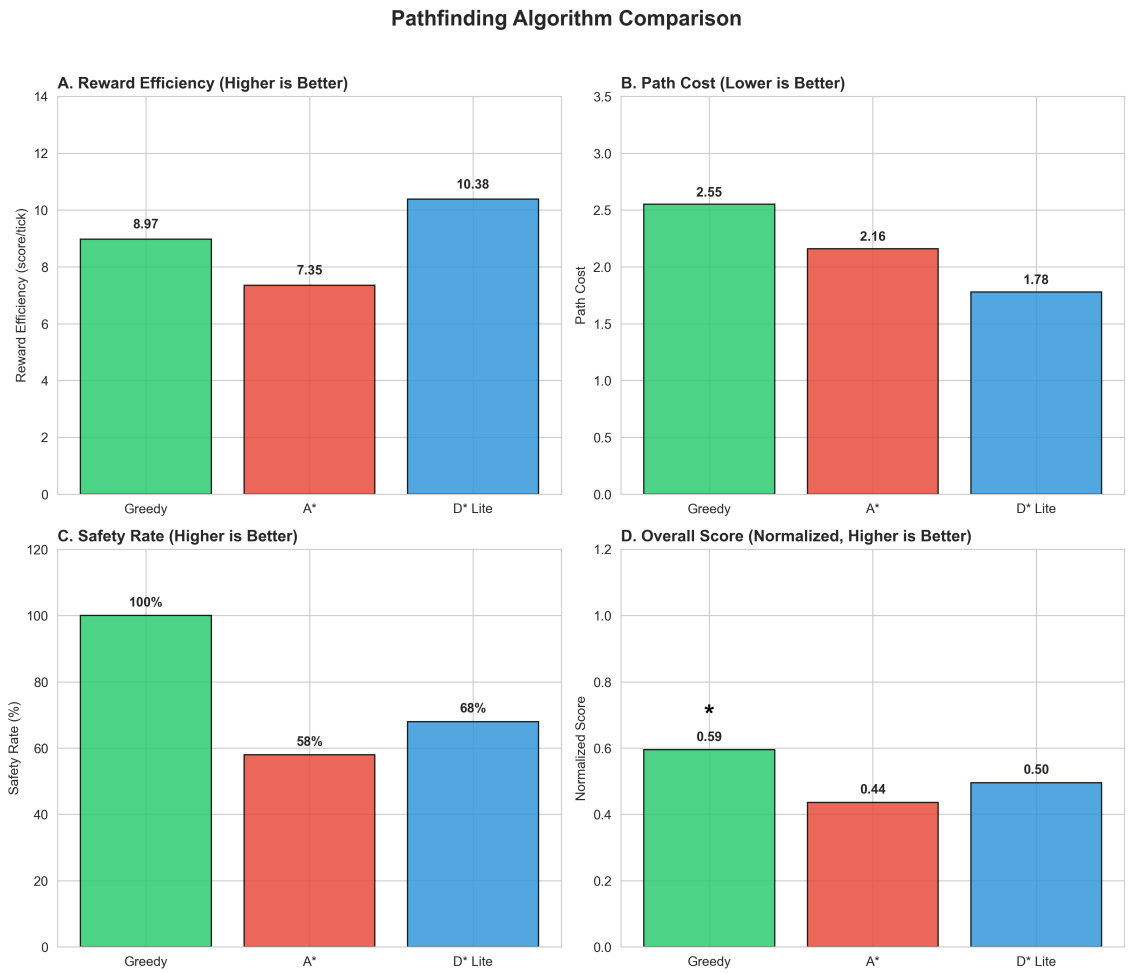

Figure 5 Multi-metric comparison of pathfinding algorithms (Greedy, A*, D* Lite).

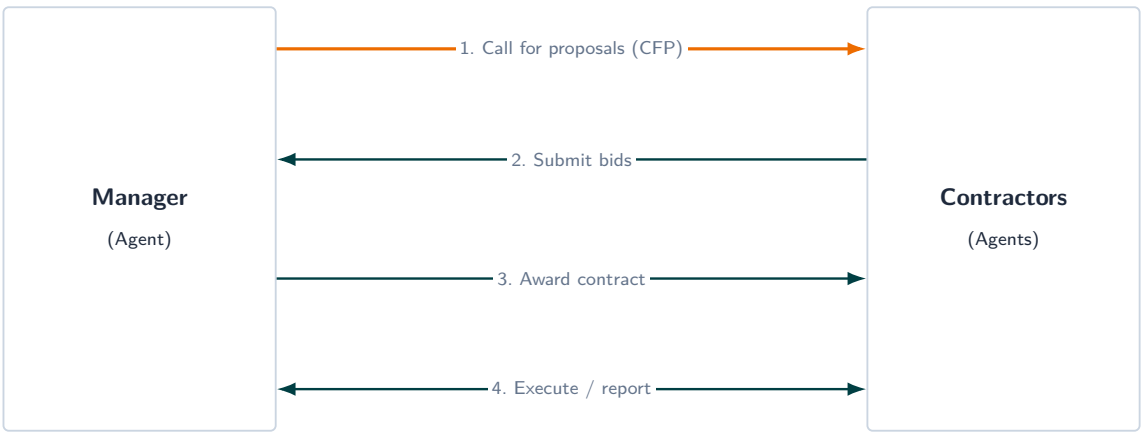

Figure 6 Contract Net handshake (conceptual)

travel cost, energy risk, and (where applicable) switching penalties—so the board can compare commitments on a common scale.

3. **Award:** The board evaluates feasible bids and awards the contract to the highest-utility bidder; ties or infeasible bids are resolved according to the configured evaluation strategy.

4. **Execution:** The awarded agent executes the task (acquire/move/fill or recharge). If execution fails or the task becomes invalid, the contract may be released and the task re-auctioned in a later tick.

6.1.2 Task Types

The implementation classifies work into three CNP task types so the board can announce the minimal commitment required—from a full hole-servicing chain down to a direct fill or a recharge slot.

Table 15 CNP task types

Task Type	Description	When Announced
`SERVICE_HOLE`	Complete task chain: acquire a tile, reach the hole, and fill it.	Agent still needs both a suitable tile and a target hole (full service contract).
`DIRECT_FILL_HOLE`	Fill the hole only; the agent already carries a tile.	Agent holds a tile and only needs routing and execution for the fill action.
`RECHARGE_SLOT`	Claim a slot at an energy station for replenishment.	Agent must recharge (proactive or critical energy state) and competes for station access.

Each row corresponds to a distinct announcement path on the board: `SERVICE_HOLE` coordinates tile acquisition and hole completion, `DIRECT_FILL_HOLE` avoids redundant tile claims when cargo is already held, and `RECHARGE_SLOT` serializes contention at scarce stations alongside hole tasks.

6.2 Contract Evaluation Strategy

The core bid evaluation is implemented in `ContractEvaluationStrategy.java`:

Java

```java
public interface ContractEvaluationStrategy {
    ContractBid buildHoleBid(Agent bidder, Hole hole, List<Tile> candidateTiles,
            Grid<Object> grid, List<EnergyStation> allStations,
            int safetyMargin, int maxGridDistance);

    ContractBid buildRechargeBid(Agent bidder, EnergyStation station,
            Grid<Object> grid, int safetyMargin, int maxGridDistance);

    int compareBids(ContractBid a, ContractBid b);
}
```

Utility formulas for hole service, direct fill, and recharge follow the same weighted structure as in R4 (score, hold bonus, distance, risk, switch penalty; recharge adds urgency and queue terms). Feasibility gates require available targets, sufficient energy, and agent not powered down.

6.3 Integration with BDI

In Contract Net mode, coordination goes through the board each tick:

Java

```java
if (this.contractNetBoard != null) {
    this.contractNetBoard.tick(this, beliefState, currentPosition);
} else {
    List<Agent> partners = this.policyBundle.getCoordinationStrategy()
            .selectPartners(this, beliefState, this.currentIntention);
}
```

6.4 Evaluation: Collaboration vs Non-Collaboration

Table 16 Coordination mode comparison (seeds 1–10, A* pathfinding)

Metric	DISABLED	PASSIVE	ACTIVE	CONTRACT_NET
Completion Time	2998.0	3063.1	3278.0	**699.8** *
Reward Efficiency	7.71	7.54	7.52	**19.48** *
Path Cost	2.14	2.14	2.17	**1.51** *
Safety Rate	58%	58%	56%	**100%** *
Remaining Holes	0.6	0.6	1.1	**0.0** *
Cooperation Gain	0.0	-2.17	-9.34	**+76.66** *

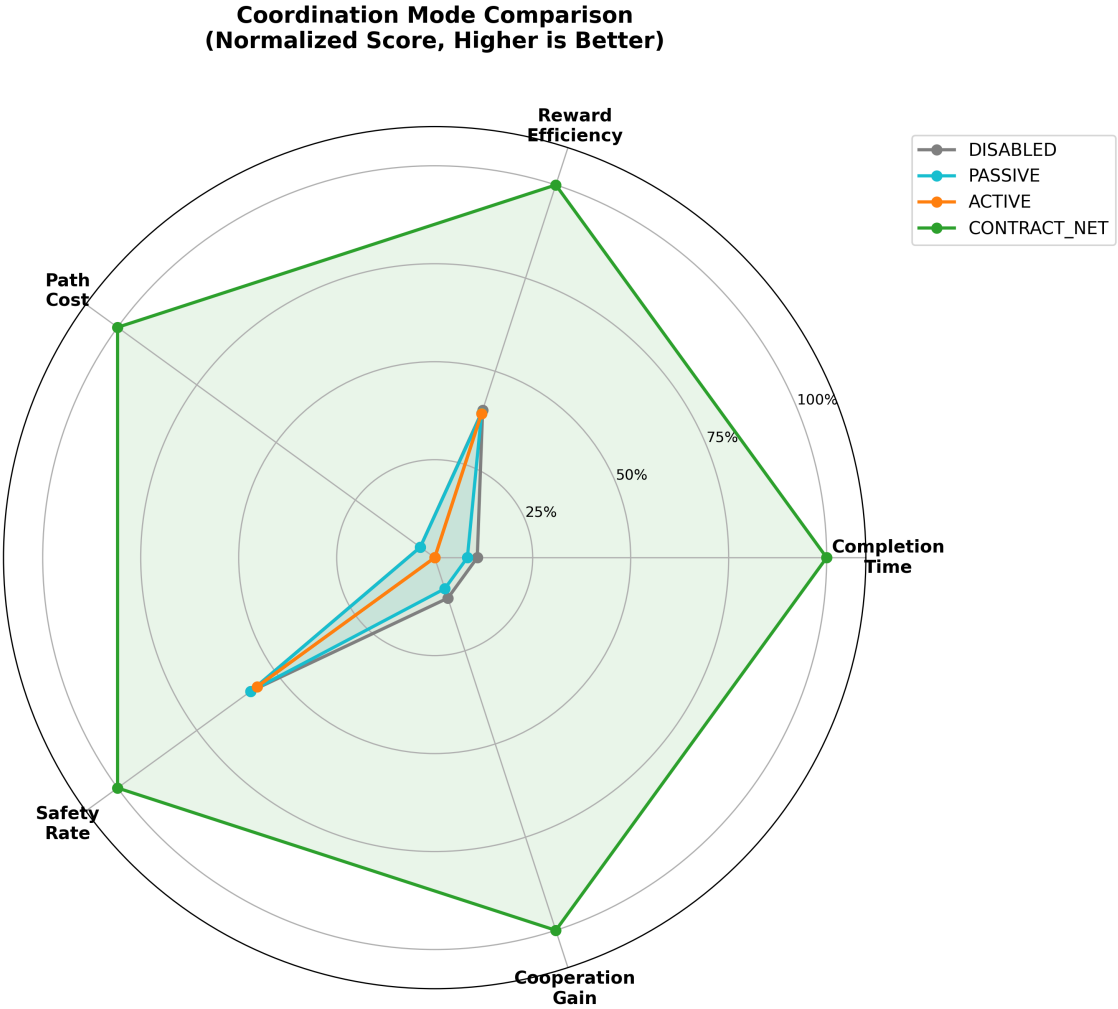

Figure 7 Radar chart comparing coordination modes across metrics. CONTRACT_NET dominates.

6.4.1 Key Observations

Contract Net yields about 4.3× faster completion, ∼ 3× reward efficiency, 100% safety, and zero remaining holes versus other modes; ACTIVE adds overhead without effective allocation.

6.5 Discussion

CNP improves efficiency by assigning high-value holes to capable nearby agents, prioritizing critical recharge, and reducing redundant work. Limitations: single-agent task chains, CNP latency under very fast wind, static bid weights.

References

[1] M. E. Pollack and M. Ringuette. Introducing the Tileworld: Experimentally Evaluating Agent Architectures. In *AAAI-90*, pages 183–189. AAAI Press, 1990.

[2] R. G. Smith. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Trans. Comput.*, C-29(12):1104–1113, 1980. https://www.reidgsmith.com/The_Contract_Net_Protocol_Dec-1980.pdf

[3] Wellshh. *PathFinderAlgo — Path Planning Algorithms Library*, 2024. https://github.com/Wellshh/PathFinderAlgo

[4] M. Wooldridge. *Reasoning about Rational Agents*. MIT Press, 2003.

A Key Code Snippets

A.1 BDI Execution Cycle (`Agent.java#run()`)

Java

```java
@ScheduledMethod(start = 1, interval = 1)
public void run() {
    if (this.poweredDown) { return; }

    AgentBeliefState preBelief = this.buildBeliefState();
    if (this.contractNetBoard == null) {
        this.reviseBeliefs(preBelief);
    }
    AgentBeliefState beliefState = this.buildBeliefState();

    List<AgentDesire> desires = this.policyBundle.getDesireEvaluationStrategy()
            .evaluate(beliefState, this);

    AgentIntention nextIntention = this.policyBundle.getTaskSelectionStrategy()
            .selectIntention(beliefState, desires, this.currentIntention);
    this.commitIntention(nextIntention, currentPosition);

    List<Agent> partners = this.policyBundle.getCoordinationStrategy()
            .selectPartners(this, beliefState, this.currentIntention);
    this.syncCoordinationPartners(partners);

    this.act(context);
}
```

A.2 Desire Evaluation (DefaultDesireEvaluationStrategy.java)

Java

```java
public List<AgentDesire> evaluate(AgentBeliefState beliefState, Agent agent) {
    List<AgentDesire> desires = new ArrayList<>();

    if (this.rechargeStrategy.shouldRecharge(beliefState)
            && beliefState.getClaimedEnergyStation() != null) {
        desires.add(new AgentDesire(AgentDesireType.RECHARGE, 100,
                beliefState.getClaimedEnergyStation()));
    }

    if (beliefState.isHoldingTile() && beliefState.getClaimedHole() != null) {
        desires.add(new AgentDesire(AgentDesireType.FILL_HOLE, 90,
                beliefState.getClaimedHole()));
    }

    if (!beliefState.isHoldingTile() && beliefState.getClaimedTile() != null) {
        desires.add(new AgentDesire(AgentDesireType.ACQUIRE_TILE, 80,
                beliefState.getClaimedTile()));
    }

    desires.add(new AgentDesire(AgentDesireType.EXPLORE, 10, null));
    return desires;
}
```

A.3 Greedy Path Planning (GreedyPathPlanningStrategy.java)

Java

```java
public GridPoint nextStep(Agent agent, GridPoint currentPosition,
        GridPoint targetPosition) {
    GridDimensions dimensions = agent.getGrid().getDimensions();
    int curX = currentPosition.getX(), curY = currentPosition.getY();

    if (curX != targetPosition.getX()) {
        int[] candidateX = this.computeNextCoordinate(
                curX, targetPosition.getX(), dimensions.getWidth());
        if (agent.canMoveTo(candidateX[0], curY))
            return new GridPoint(candidateX[0], curY);
        if (agent.canMoveTo(candidateX[1], curY))
            return new GridPoint(candidateX[1], curY);
    }

    if (curY != targetPosition.getY()) {
        int[] candidateY = this.computeNextCoordinate(
                curY, targetPosition.getY(), dimensions.getHeight());
        if (agent.canMoveTo(curX, candidateY[0]))
            return new GridPoint(curX, candidateY[0]);
        if (agent.canMoveTo(curX, candidateY[1]))
            return new GridPoint(curX, candidateY[1]);
    }

    return currentPosition;
}
```

A.4 Wind Dynamic Update (`Wind.java#blow()`)

Java

```java
public void blow() {
    List<Obstacle> obstacles = new ArrayList<>();
    List<Tile> tiles = new ArrayList<>();
    for (Object obj : this.context) {
        if (obj instanceof Obstacle) obstacles.add((Obstacle) obj);
        else if (obj instanceof Tile) tiles.add((Tile) obj);
    }

    List<int[]> candidateCells = this.collectCandidateCells();
    this.pickWithoutReplacement(candidateCells, obstacles.size() + tiles.size());

    int cursor = 0;
    for (Obstacle o : obstacles) {
        int[] pos = candidateCells.get(cursor++);
        this.grid.moveTo(o, pos[0], pos[1]);
    }
    for (Tile t : tiles) {
        int[] pos = candidateCells.get(cursor++);
        this.grid.moveTo(t, pos[0], pos[1]);
    }
}
```